

Skin-first beauty recommender

A user states their skin type, tone, concerns, category and budget. The system returns ranked products, each carrying the evidence behind it — including the criteria a product *fails*. No explanation is generated by a language model.

LIVE APP	REPOSITORY	CATALOGUE	INTERACTIONS	TESTS	COMMIT
orboai.streamlit.app	gkbishnoi07/Orbo.AI	8,494 products	893,393	187 passing	9046c5f

CONTENTS

01	What I built	07	How to run it
02	Why this domain	08	Evaluation
03	How it works	09	Test cases
04	Architecture	10	Limitations
05	Recommendation approach	11	Future work
06	Dataset		

What I built

A deployed web application that turns a stated skin profile into a short ranked product list with an auditable reason attached to every item. It runs on 8,494 real Sephora products and 893,393 real positive ratings from people who declared their own skin type.

Four scoring layers sit behind a hard filter stage: content similarity over product text, cohort-scoped collaborative filtering, a popularity prior, and a skin-tone affinity nudge. Results are then reranked for variety and passed through a template-driven explanation layer.

THE DESIGN POSITION

The half of this problem that interested me was not ranking — it was **auditability**. A beauty recommendation a shopper cannot check is a guess with a UI attached. Every claim on every card is read from a column, so the system can say what a product *fails* as readily as what it satisfies, and it cannot invent an ingredient claim.

Why this domain

I chose a beauty and skincare recommender because Orbo.AI works in this domain — visual skin analysis, shade matching, product recommendation — so the design decisions here are the same ones the company's own products have to make. The easier option was a MovieLens or generic e-commerce dataset, which is what most tutorials use. I picked beauty because it makes the problem harder in three specific ways.

- **Suitability is not preference.** A fragranced serum is not a weaker match for reactive skin; it is the wrong answer. Some constraints must override the score entirely, which forces a filter layer genuinely separate from ranking.
- **Cohorts mean something.** “People with oily skin who liked this also liked that” is a different statement from the unscoped version. This dataset carries reviewer skin type, so the idea is testable rather than aspirational.
- **Explanations carry real stakes.** People put these products on their face. An invented claim about an ingredient is not a bad recommendation, it is a safety problem — which is why the explanation layer is template-driven and an LLM is deliberately absent.

How it works

Two phases. Heavy work happens once, offline, and its output is committed to the repository. The deployed app reads only those files, which is why it needs no Kaggle credentials, never parses a CSV, and never loads `torch`.

OFFLINE

Download & audit

`scripts/00_download.py` fetches the Kaggle CSVs. `01_inspect.py` audits them and decides whether cohort CF is viable at all — the architecture was not fixed until that verdict came back.

OFFLINE

Vectorise & build

`02_embed.py` builds TF-IDF and MiniLM vectors so the two can be compared.

`03_build_artifacts.py` writes the 33 MB the runtime reads.

REQUEST

Hard filters

Budget, category, and suitability rules. Applied before anything is ranked, in the `Recommender` base class so every strategy — baselines included — honours them identically.

REQUEST

Score, blend, rerank

Four layers scored, min-max scaled, weighted, then greedy MMR at $\lambda = 0.85$ for intra-list variety.

REQUEST

Explain

Each result is turned into evidence lines grouped as *why it matches*, *evidence*, and *worth knowing*. Templates filled from columns, never generated.

MEASURED

Model inference p95 is **12.2 ms**. End-to-end recommendation — inference plus explanation generation at $k = 20$ — is **16.2 ms p50, 30.9 ms p95**. Neither figure includes browser rendering. These are two different numbers and the documentation names which is which everywhere it quotes one.

Architecture

```

OFFLINE (run once, output committed)
  scripts/00_download.py      --> data/raw/          530 MB Kaggle CSVs, gitignored
  scripts/01_inspect.py      --> reports/data_audit.md the audit that fixed the design
  src/data.py                --> canonical frames    ONLY module that knows the
                                                    raw Kaggle column names

  scripts/02_embed.py        --> TF-IDF + MiniLM vectors
  scripts/03_build_artifacts.py --> artifacts/      33 MB, committed to the repo

RUNTIME (per request, ~12 ms)
  Query --> _apply_filters --> [ content      ]
                                [ budget      ]
                                [ category    ] --> blend --> MMR --> Explainer --> cards
                                [ rules       ]

```

Module boundaries

`src/schema.py` is the contract: canonical column names, `Query`, `Scored`, `Evidence`. Everything downstream of `src/data.py` speaks canonical names only, so swapping the dataset touches exactly one file. `src/service.py` is the composition root shared by the UI and the scripted test cases, so the two cannot disagree about what “the hybrid” means.

13 runtime modules, 7 offline scripts, 9 test files	
MODULE	RESPONSIBILITY
<code>schema.py</code>	The contract — names, types, vocabulary, thresholds
<code>data.py</code>	Only module aware of raw Kaggle column names
<code>rules.py</code>	Hard suitability rules, precomputed per catalogue
<code>recommender.py</code>	Base class + random and popularity baselines
<code>content.py</code>	Content scoring over precomputed vectors
<code>collaborative.py</code>	Cohort-scoped item-item CF
<code>tone.py</code>	Skin-tone affinity, shrunk and signed
<code>hybrid.py</code>	Weighted blend + MMR reranker
<code>explain.py</code>	Template-based evidence lines
<code>metrics.py</code>	Pure metric functions, no dataset dependency
<code>evaluate.py</code>	Leave-one-out harness, <code>compare()</code>
<code>artifacts.py</code>	Loads the committed runtime files
<code>service.py</code>	Composition root for UI and test cases

Recommendation approach

Stage 1 — hard filters

Two rules, both derived from the profile rather than from a score. **Irritant exclusion:** declaring sensitivity removes products whose ingredient list contains fragrance, drying alcohol, fragrant essential oils, or a harsh sulfate surfactant. The alcohol patterns are deliberately specific — `alcohol denat`, `sd alcohol` — because a bare `alcohol` match would also catch cetyl and stearyl alcohol, which are fatty alcohols and emollients, the opposite of an irritant.

Stated skin-type mismatch: a product labelled “Best for Oily, Combo, Normal” is excluded for dry skin.

THE MOST CONSEQUENTIAL DECISION IN THE CODEBASE

Only **12.4%** of products carry any skin-type tag. So the rule fires on *stated disagreement*, never on a missing tag. Treating untagged as unsuitable would silently delete seven eighths of the catalogue while still returning a confident-looking list of ten. Three separate tests assert this.

Stage 2 — four scoring layers

Content

0.15 / 0.10

Cosine similarity to a profile prototype built from catalogue centroids, plus exact concern and skin-type tag matching. The only layer that can rank a product nobody has reviewed — applicable to the 72% of the catalogue with no interactions.

Cohort CF

0.80 / 0.30

Item-item cosine similarity with co-occurrence shrinkage, rebuilt per skin-type cohort. Captures what product copy never says: which foundation oxidises, which moisturiser pills under sunscreen.

Popularity

0.05 / 0.60

`log1p` of the count of ratings at 4+. Breaks ties toward things people actually buy, and carries the cold-start case where little else is known.

Tone affinity

+0.10

A band's positive rate minus the product's overall rate, shrunk by an empirical-Bayes prior of 20 reviews. Signed and centred on zero, added on top of the swept weights so a tone-less query is bit-identical to the three-layer version.

Weights differ by regime because cold start is a different problem: **0.15 / 0.80 / 0.05** for a returning user, **0.10 / 0.30 / 0.60** for a new one. Both were chosen from a documented sweep, not guessed.

Stage 3 — explanations

Templates filled from columns. A cohort percentage — “86% of dry-skin reviewers rated this 4 or above” — is shown only when that cohort has at least **30** reviews, and always with its sample size. Below the threshold it is suppressed rather than hedged, because a percentage from eleven reviewers reads as authoritative whatever caveat is attached. Only 51.8% of cohorts clear that bar, so roughly half of all explanations use the checklist form by design.

Unmet criteria are shown, not hidden. A checklist of nothing but ticks is marketing.

Dataset

Sephora Products and Skincare Reviews (Kaggle, 2023 snapshot). Chosen because it is the only readily available beauty dataset carrying *reviewer skin type and skin tone* — without which cohort-scoped collaborative filtering is not possible.

Every figure below is computed live by <code>src/data.py</code> , not transcribed	
Products	8,494 · 304 brands · 42 categories
Reviews	1,094,411 raw → 1,088,886 after de-duplication
Positive interactions (4+)	893,393
Distinct reviewers	430,091 with a positive
Products with any review	2,343 — 27.6% of catalogue
Matrix density	0.089%
Products with a skin-type tag	12.4%
Products with a concern tag	41.7%
Cohorts clearing $n \geq 30$	51.8%

Three properties that shaped the code

1. **There is no product description field.** The text behind the vectors had to be assembled from brand, name, categories, highlights and the ingredient list.
2. **highlights and ingredients are stringified Python lists**, not delimited text, so they need `literal_eval` rather than `split`.
3. **82% of reviews are 4★ or 5★.** The positive threshold of 4.0 is therefore generous and the task correspondingly easier than on a balanced set. Kept anyway, because moving it after seeing results would be choosing the threshold that flatters the model.

Where the data lives in the repo

If you are looking for a dataset file, it is `artifacts/` — not `data/`, which is deliberately empty. `data/raw/` is gitignored: the five original CSVs are ~530 MB and carry Kaggle's licence terms, so they are reproduced by `scripts/00_download.py` rather than redistributed. What *is* committed is 33 MB of derived data — the product table, positive interactions, per-cohort and per-tone counts, and the product vectors. That is why a clone runs with no Kaggle account.

How to run it

Use the deployed app

orboai.streamlit.app — no account, no setup.

Run locally

Requires Python **3.10 or newer**. On Windows use `.venv\Scripts\` in place of `.venv/bin/`.

```
git clone https://github.com/gkbishnoi07/Orbo.AI.git
cd Orbo.AI
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
.venv/bin/streamlit run app.py
```

Then open `http://localhost:8501`. That is all — the committed artifacts mean no dataset download is needed. First load takes a few seconds while the cohort similarity matrices are built; they are cached afterwards.

Run the tests, or reproduce everything

```
.venv/bin/pip install -r requirements-dev.txt
.venv/bin/python -m pytest -q                                # 187 tests

# full pipeline from raw data — needs ~/.kaggle/kaggle.json
.venv/bin/python scripts/00_download.py
.venv/bin/python scripts/01_inspect.py data/raw
.venv/bin/python scripts/02_embed.py
.venv/bin/python scripts/03_build_artifacts.py
.venv/bin/python scripts/04_evaluate.py
.venv/bin/python scripts/05_sweep.py
.venv/bin/python scripts/06_test_cases.py
```

VERIFIED

A clean clone from GitHub plus `requirements.txt` resolves to **41 packages with zero** of `torch`, `transformers` or `sentence-transformers`, and boots and answers its first query in **2.8 s**.

Evaluation

Leave-one-out on positive interactions. Where timestamps exist the *most recent* like is held out, so the model predicts forward in time rather than interpolating between known likes.

THE PART THAT MATTERS MORE THAN ANY SINGLE NUMBER

Every model is fitted on the same training split, with all evaluated interactions removed. Fitting a popularity or CF model on all reviews and then evaluating leave-one-out lets each model count the very interaction it is asked to predict. The leak is easy to miss and flatters interaction-based models specifically, so its size is measured rather than asserted away: training on everything inflates CF's NDCG@10 by **3.7%** and popularity's by **1.9%**.

Two case sets are reported separately because they measure different products. **Warm start** uses users with two or more positives and tests the ranking model. **Cold start** uses users with exactly one positive, hidden — the path a new visitor actually takes. Averaging them would hide the second either way.

NDCG@10, 1,000 warm and 1,000 cold held-out likes, commit <code>8116ea1</code> . Latency is model inference only.					
MODEL	WARM	COLD	COVERAGE _{WARM}	DIVERSITY _{WARM}	P95
random	0.0004	0.0000	68.8%	0.904	7.6 ms
popularity	0.0307	0.0693	0.2%	0.874	6.9 ms
content (MiniLM)	0.1166	0.0024	11.0%	0.646	10.4 ms
content (TF-IDF)	0.2404	0.0019	16.6%	0.521	9.8 ms
cohort CF	0.4135	0.0693	15.5%	0.748	7.2 ms
hybrid (TF-IDF)	0.4112	0.0653	14.2%	0.740	12.2 ms

“Coverage” here means one specific thing and it is easy to confuse with another: the share of the catalogue a model actually *returned* across the run. It is not the share a layer is *able* to score. The content layer is applicable to 72% of the catalogue (reach) while returning 16.6% of it across 1,000 cases (output). Both are about the content model; neither contradicts the other.

Three findings, including the unflattering one

TF-IDF beat MiniLM by roughly 2× for content scoring (0.2404 against 0.1166). With no description field, the product text is highlight tokens and INCI ingredient lists — keyword soup, where exact term overlap does more work than semantic similarity. Both were built and measured rather than assumed, which is the only reason I can say this rather than argue it.

STATED PLAINLY

The hybrid does not beat cohort CF alone (0.4112 against 0.4135), and the weight sweep is blunter still: on NDCG the optimum is degenerate — pure CF warm, pure popularity cold. This is largely a limitation of the protocol. Held-out items are drawn from the ratings table, so *every correct answer is a product that already has reviews*; the 6,151 unrated products can never register as a hit, and the content layer's whole contribution is invisible to the measurement. The shipped weights keep small content terms anyway, at a stated cost of −0.5% warm and −6% cold NDCG.

Cold start is genuinely weak. Nothing beats plain popularity by much. Said here rather than buried.

Metrics: precision@10, recall@10, NDCG@10, MAP@10, recommendation coverage, intra-list diversity, novelty, p50/p95 latency, empty-result rate. Coverage and diversity are reported *beside* accuracy because a model that serves every user the same ten bestsellers can score well on accuracy alone.

Test cases

Ten scenarios — **five success, five failure** — generated by `scripts/06_test_cases.py` running the shipped service, with real output tables in `reports/test_cases.md`. Generated rather than transcribed, because a hand-written table of expected output drifts from the code the first time a weight changes and nobody notices.

#	SCENARIO	WHAT IT EXERCISES
S1	Dry + sensitive, moisturiser, <\$60	Core path with a hard rule attached
S2	Oily + acne	Concern tag genuinely steers the list
S3	Returning shopper, 3 owned products	Warm start; item-item CF actually fires
S4	Hair concern, no category chosen	Reaches haircare without manual filtering
S5	No profile at all	Honest popularity, not fake personalisation
F1	Budget below category minimum	Zero results, constraints named — no silent relaxation
F2	Cold start, skin type only	Measurably no better than popularity
F3	Every concern selected at once	Concern signal dilutes to nothing
F4	Sensitivity + tight budget	Compounding rules collapse the eligible pool
F5	Skin tone stated	Documents exactly how weak the tone signal is

The suite itself is **187 tests** across 9 files: metric arithmetic pinned to values computed by hand, the leak-prevention path, the MMR diversity trade, suppression of thin cohort statistics, and 40+ tests driving the real Streamlit page through `AppTest` — because a UI that silently ignores a control is the failure mode worth guarding, and it is invisible to a screenshot.

Limitations

1. **Cold start barely beats popularity** (0.0653 against 0.0693). With a profile and no history there is little to personalise on.
2. **The hybrid loses to CF alone on offline accuracy**, and the offline metric structurally cannot see what the content layer adds.
3. **CF is blind to 72% of the catalogue**. Only 2,343 products have any review.
4. **Skin tone is a weak signal, deliberately**. It affects ranking but visibly moves results only for the *deep*, *medium* and *tan* bands — *fair* and *light* dominate the data, so their rate already *is* the average everything is shrunk toward. True shade matching remains impossible: this dataset has no product-level shade data.
5. **MMR's measured benefit is replicable by a simpler rule**. At $\lambda = 0.85$ it reaches 0.8372 intra-list diversity; a naive one-product-per-brand cap reaches the same figure. MMR is kept because it generalises to similarity the brand field does not capture, not because the measurement favours it.
6. **The 4★ threshold is generous** — 82% of reviews qualify.
7. **Profile fields are self-reported and unverified**, and in reality skin changes seasonally.
8. **No feedback loop**. Everything is offline; no clicks, so nothing learns from use.
9. **A 2023 snapshot**. Prices and rating counts were true when scraped, not today.
10. **Popularity bias survives in CF**. Item-item similarity on sparse retail data returns blockbusters as almost everyone's neighbour — the median popularity rank of a top-50 neighbour is 96 of 8,494. MMR mitigates, it does not solve.
11. **The result list is capped at 100**. Each card is roughly eight Streamlit elements, so the full eligible set would be ~45,000 elements and the page would be unusable.
12. **English-language, US catalogue, USD pricing**. Prices are the catalogue's own; converting would invent a figure at a rate that goes stale immediately.

Future work

In priority order, judged by expected value rather than by what is interesting to build.

1. **An online test.** Not a model — a measurement. Offline leave-one-out cannot distinguish the hybrid from CF-only in the way that matters, because held-out items always come from the ratings table. A week of click-through data would settle what no amount of offline tuning can. Everything below is guesswork until this exists.
 2. **Structured ingredient parsing.** Ingredients are currently a text blob truncated at 400 characters. Parsing INCI names into a vocabulary with positional weighting would allow genuine actives matching — “contains niacinamide in the top third of the list” — and would sharpen the sensitivity rules from keyword matching into something a dermatologist would recognise.
 3. **Fix cold start properly.** Replace centroid similarity with max-similarity to any tag-matched product. A centroid over 500 products washes out into “how average is this item”; nearest-neighbour to the relevant set is far more discriminative. The most promising unexplored lever on the weakest path.
 4. **A learned reranker.** Replace the hand-tuned linear blend with a small GBDT over the component scores. The sweep showing a degenerate linear optimum is usually a sign the combination function is too simple, not that a layer is worthless.
 5. **Routine construction.** Take the ranked list and solve for a compatible set — no conflicting actives, covering cleanse/treat/moisturise inside a budget. A constrained selection problem on top of ranking, and closer to what a shopper actually wants.
 6. **Shade matching,** which is where computer vision would add something reviews cannot — but it needs shade-level product data this dataset lacks.
 7. **Calibrated confidence.** The system already knows when it is guessing: untagged product, cohort under 30, no history. It should say so in words rather than only omitting a claim.
-

Bonus comparison against Sephora and Nykaa — similarities, differences, a limitations table and next steps — is in `docs/comparison.md`. It compares only observable product behaviour; it does not speculate about either company's internal systems.

Every figure in this document is computed from the committed artifacts or read from `reports/evaluation.json`, and is reproducible with the commands in section 07.